BDD by any other name

Many of the ideas around BDD are not new and have been practiced for many years under a number of different names. Some of the more common terms used for these practices include Acceptance-Test-Driven Development, Acceptance Test-Driven Planning, and Specification by Example. To avoid confusion, let's clarify a few of these terms in relation to BDD.

Specification by Example describes the set of practices that have emerged around using examples and conversation to discover and describe requirements. In his seminal book of the same name,[14] Gojko Adzic chose this term as the most representative name to refer to these practices. Using conversation and examples to specify how you expect a system to behave is a core part of BDD, and we'll discuss it at length in the first half of this book.

Acceptance-Test-Driven Development (ATDD) is now a widely used synonym for Specification by Example, but the practice has existed in various forms since at least the late 1990s. Kent Beck and Martin Fowler mentioned the concept in 2000,[15] though they observed that it was difficult to implement acceptance criteria in the form of conventional unit tests at the start of a project. But unit tests aren't the only way to write automated acceptance tests, and since at least the early 2000s, innovative teams have asked users to contribute to executable acceptance tests and have reaped the benefits.[16]

Acceptance-Test-Driven Planning is the idea that defining acceptance criteria for a feature leads to better estimates than doing a task breakdown.

1.3.3 BDD principles and practices

Today BDD is successfully practiced in a large number of organizations of all sizes around the world, in a variety of different ways. In *Specification by Example*, Gojko Adzic provides case studies for over 50 such organizations. In this section, we'll look at a number of general principles or guidelines that BDD practitioners have found useful over the years.

Figure 1.6 gives a high-level overview of the way BDD sees the world. BDD practitioners like to start by identifying business goals and looking for features that will help deliver these goals. Collaborating with the user, they use concrete examples to illustrate these features. Wherever possible, these examples are automated in the form of executable specifications, which both validate the software and provide automatically updated technical and functional documentation. BDD principles are also used at the coding level, where they help developers write code that's of higher quality, better tested, better documented, and easier to use and maintain.

---

[14] Gojko Adzik, *Specification by Example* (Manning, 2011).

[15] Kent Beck and Martin Fowler, *Planning Extreme Programming* (Addison-Wesley Professional, 2000).

[16] Johan Andersson et al., "XP with Acceptance-Test Driven Development: A Rewrite Project for a Resource Optimization System," *Lecture Notes in Computer Science Volume* 2675 (2003). Available at http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.108.6097&rep=rep1&type=pdf.